EXPERIMENT 7

Aim- Implement the embedding wallet (Metamask) and transaction using Solidity.

Theory-

What is MetaMask?
MetaMask is a cryptocurrency wallet and browser extension that allows users to interact with blockchain networks like Ethereum.

- It stores private keys securely
- Enables sending/receiving crypto (like ETH)
- Allows interaction with decentralized applications (DApps)
- Works as a bridge between your browser and

blockchain What is Testnet?

A test net (test network) is a blockchain environment used for testing smart contracts and transactions without using real money.

- Uses free “test cryptocurrency” (no real value)
- Helps developers debug and experiment safely
- Examples:
    a. Sepolia Testnet
    b. Goerli Testnet

List the steps to connect MetaMask with RemixIDE for transactions.

Step 1: Install MetaMask

- Add MetaMask extension to browser
- Create or import a wallet

Step 2: Select a Network

- Open MetaMask
- Choose a test network (e.g., Sepolia)

Step 3: Get Test ETH

- Use a faucet to receive free test ETH

Step 4: Open Remix IDE

- Go to Remix IDE in browser

Step 5: Write/Compile Smart Contract

- Create a Solidity file
- Compile the contract

Step 6: Connect MetaMask to Remix

- Go to Deploy & Run Transactions tab
- In Environment dropdown → select Injected Provider - MetaMask

Step 7: Authorize Connection

- MetaMask will pop up → click Connect

Step 8: Deploy Contract

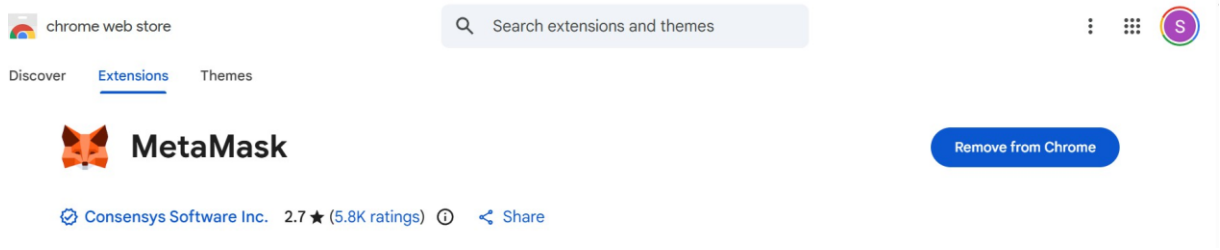
- Click Deploy in Remix
- Confirm transaction in MetaMask

Step 9: Perform Transactions

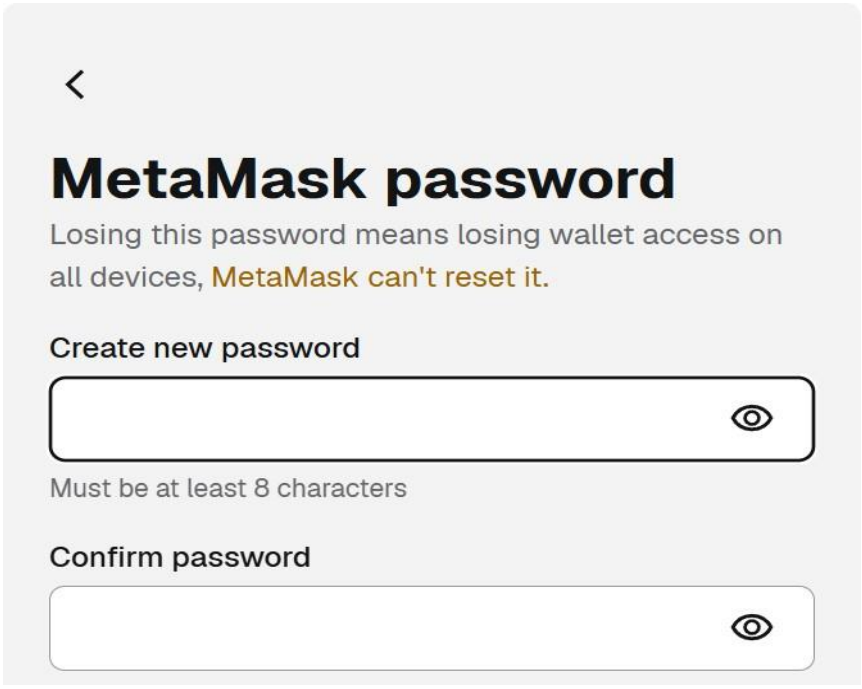
- Call contract functions
- Each transaction will require MetaMask confirmation

Implementation-

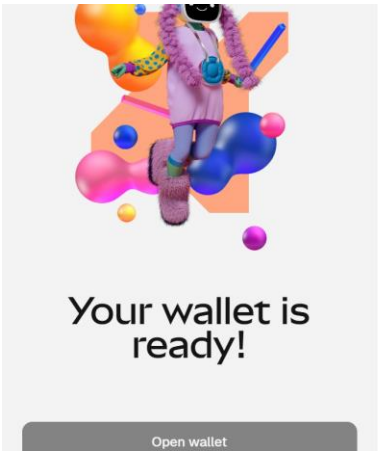
Install metamask

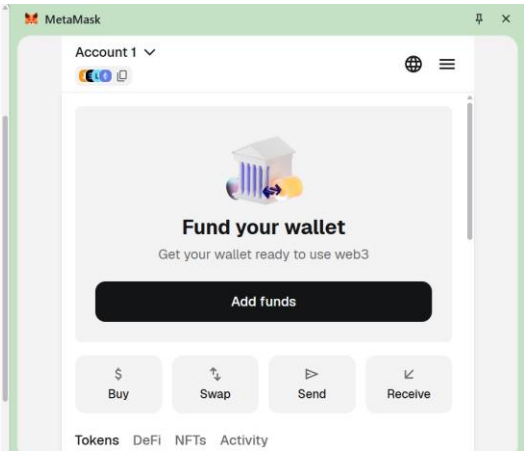


Create a new metamask wallet and give it a password



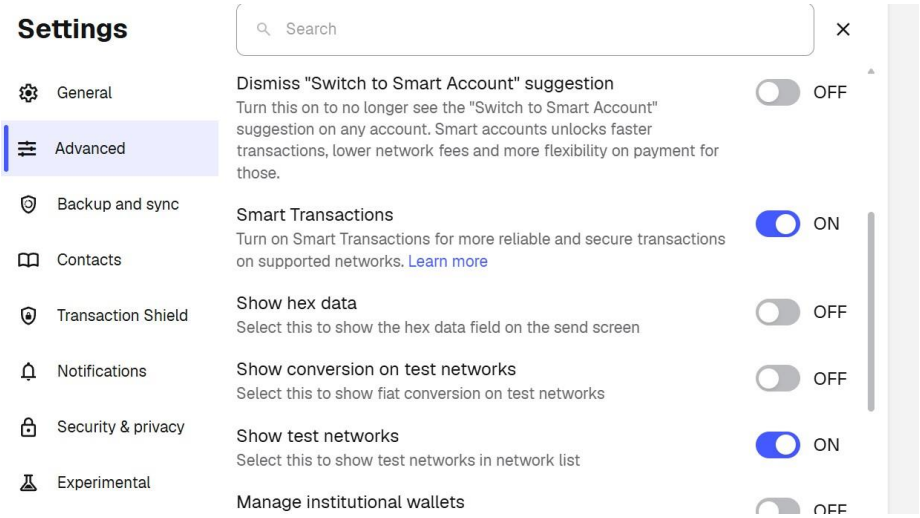
Account is successfully created



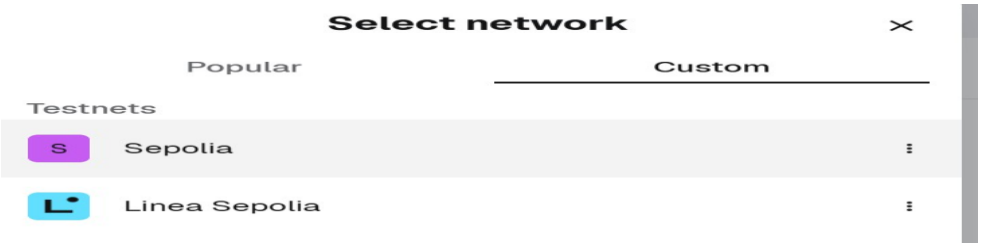


Steps to get funds from Sepolia Testnet

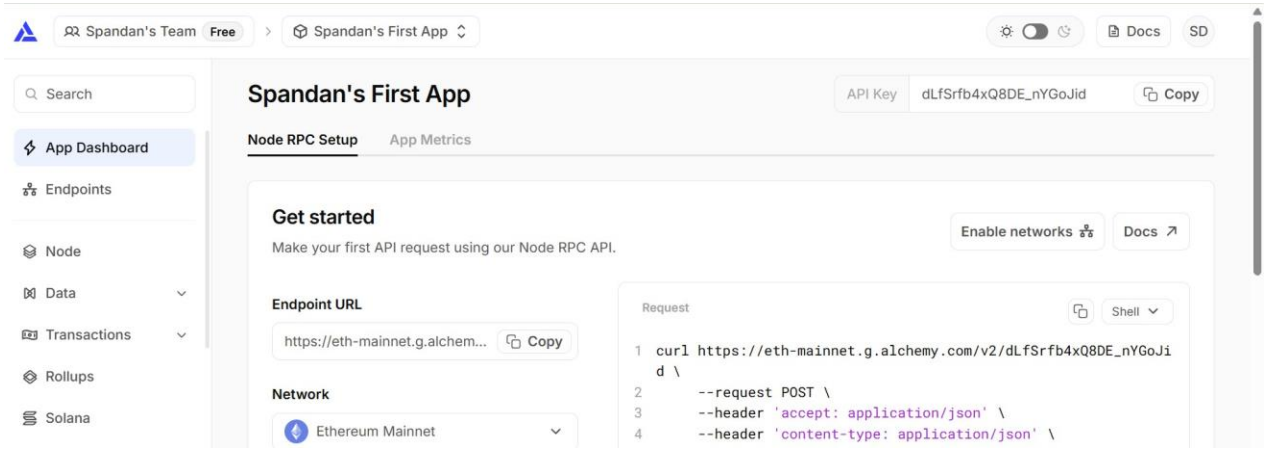
In settings switch on to show test networks



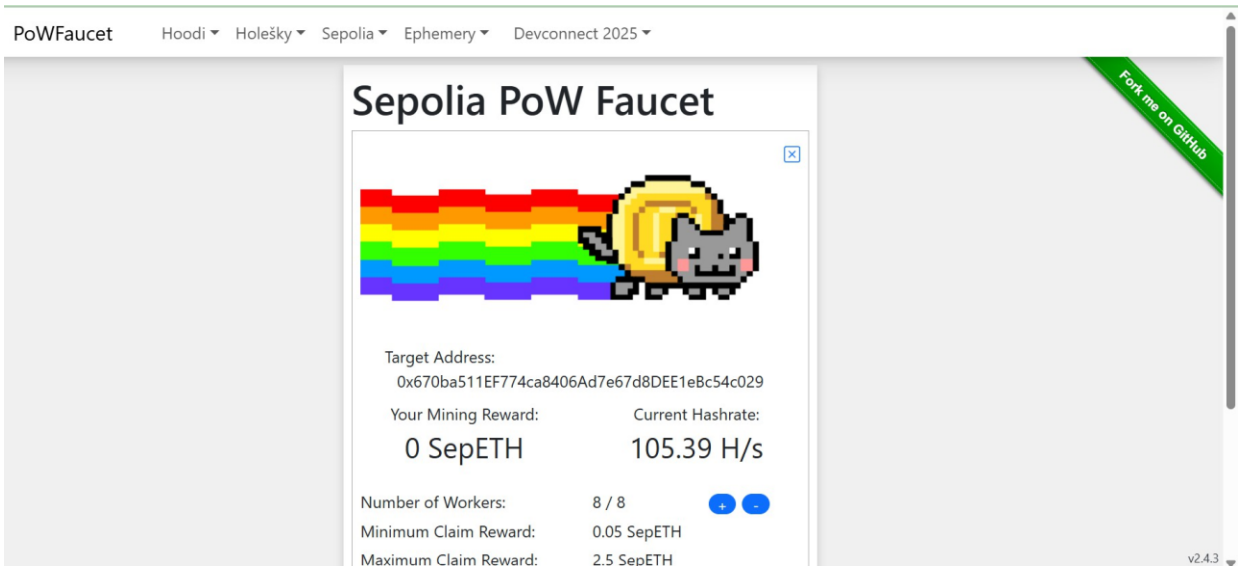
Select sepolia testnet



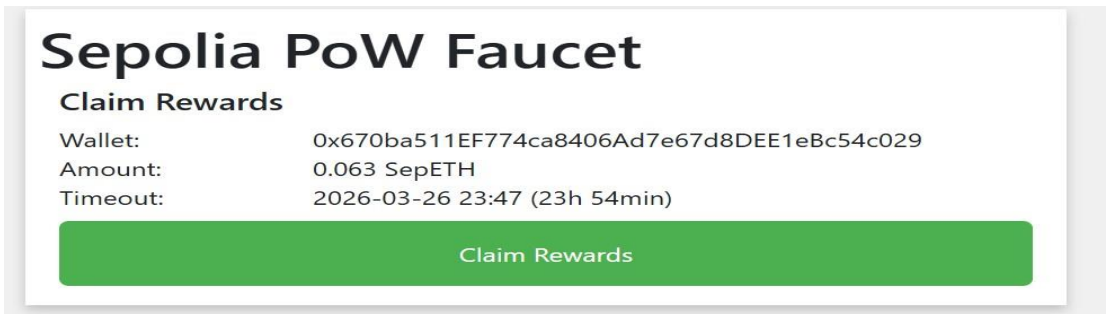
Create a free account in alchemy



Alchemy now requires this at least 0.001 ETH on Ethereum Mainnet,Shifting to PoW faucet.

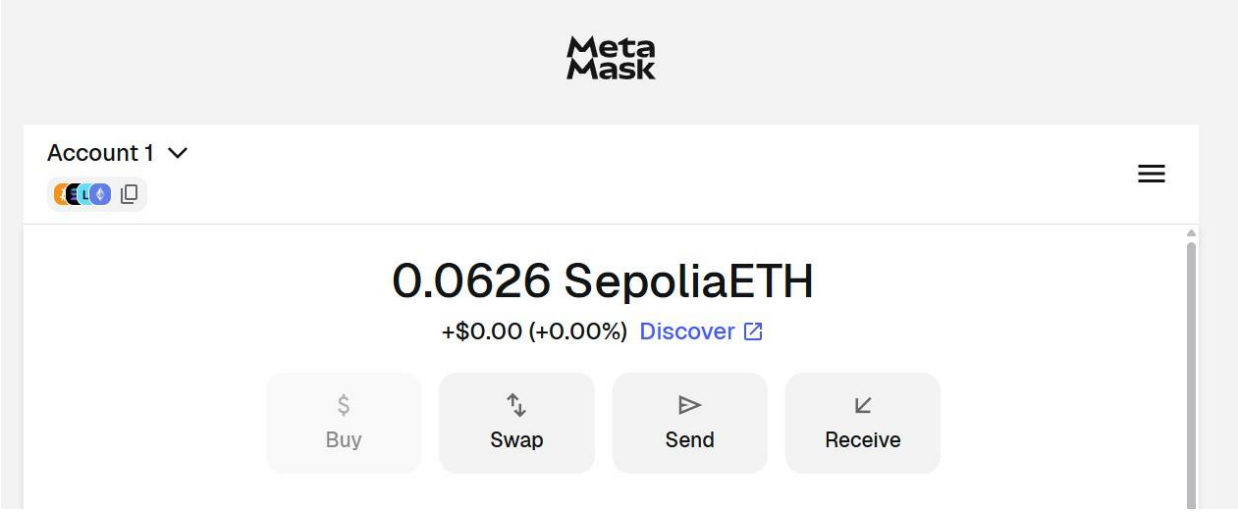


Click on Claim rewards

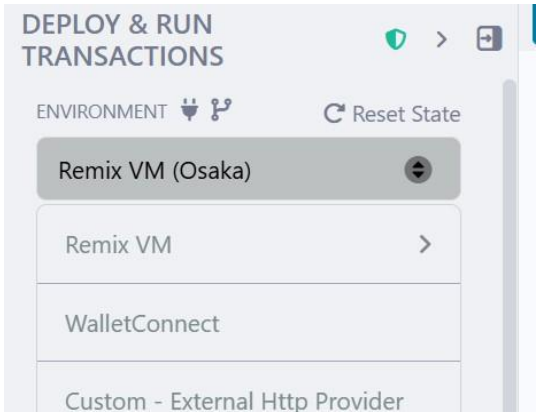


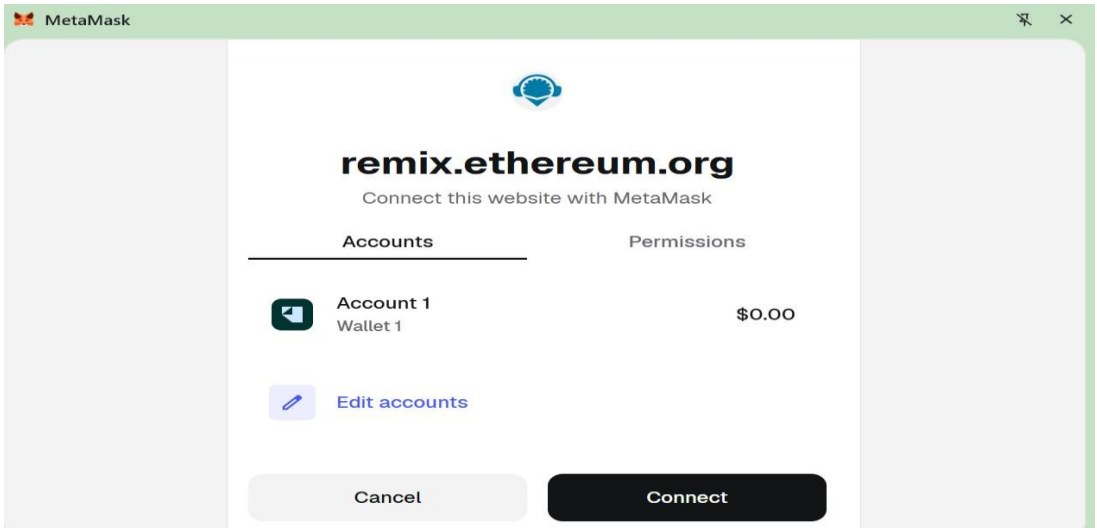
Transaction is confirmed

Account Balance



Go to Remix IDE ,compile the smart contract in deploy under environment select wallet connect



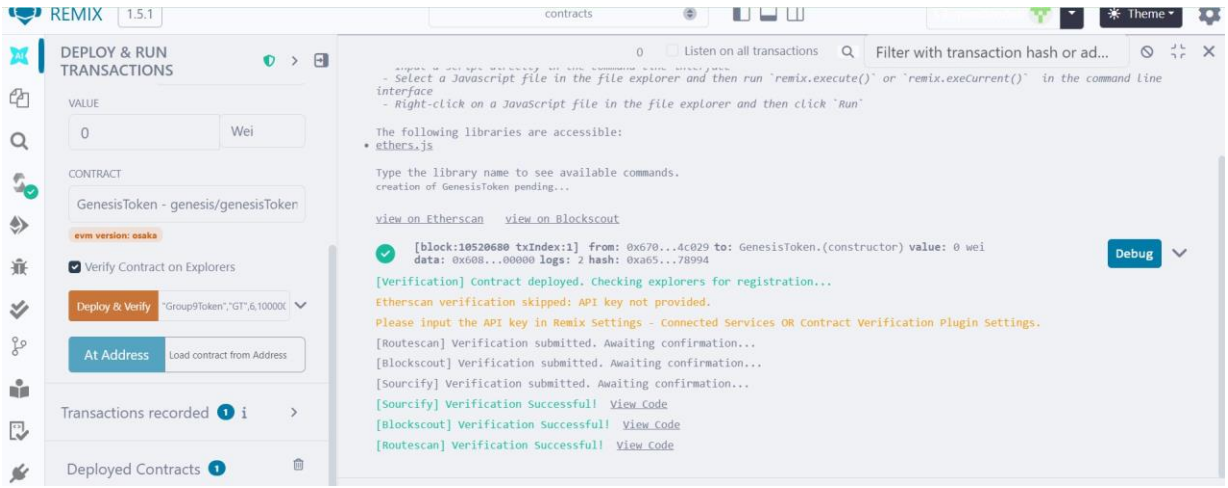


GenesisToken.sol

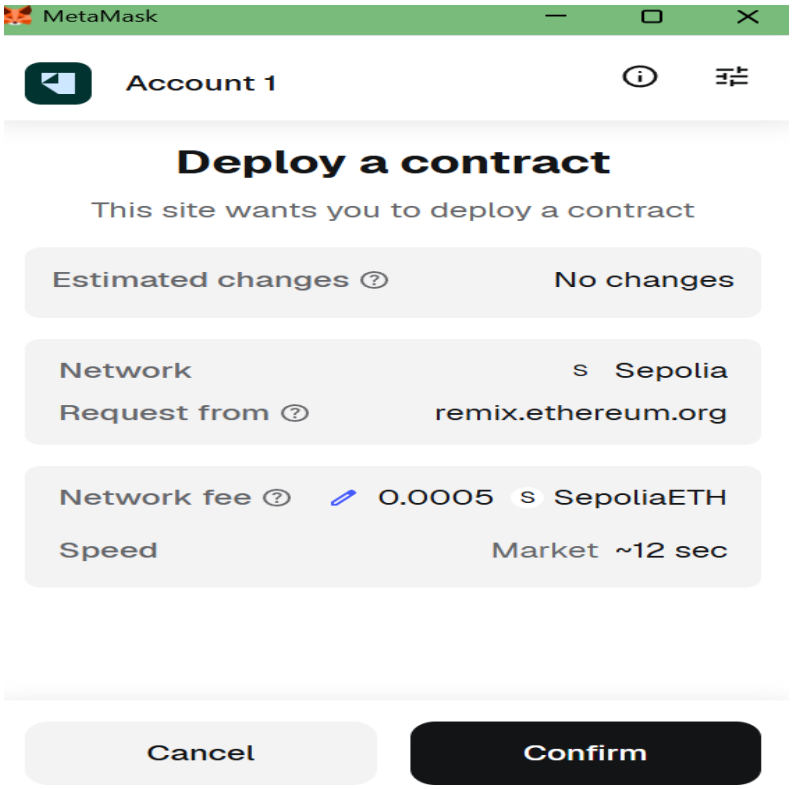
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
/**
 * @title GenesisToken
 * @notice Simple ERC20 used as the monitored token in Genesis demos.
 *         Owner can mint tokens for testing transfers, whale movements, etc.
 */
contract GenesisToken is ERC20, Ownable {
    uint8 private _decimals;

    constructor(
        string memory name_,
        string memory
        symbol_, uint8
        decimals_, uint256
        initialSupply
    ) ERC20(name_, symbol_) Ownable(msg.sender) {
        _decimals = decimals_;
        _mint(msg.sender, initialSupply * (10 ** decimals_));
    }
```

```
    function decimals() public view override returns (uint8) {
        return _decimals;
    }

    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }

    function burn(uint256 amount) external {
        _burn(msg.sender, amount);
    }
}
```

Contract Deployed

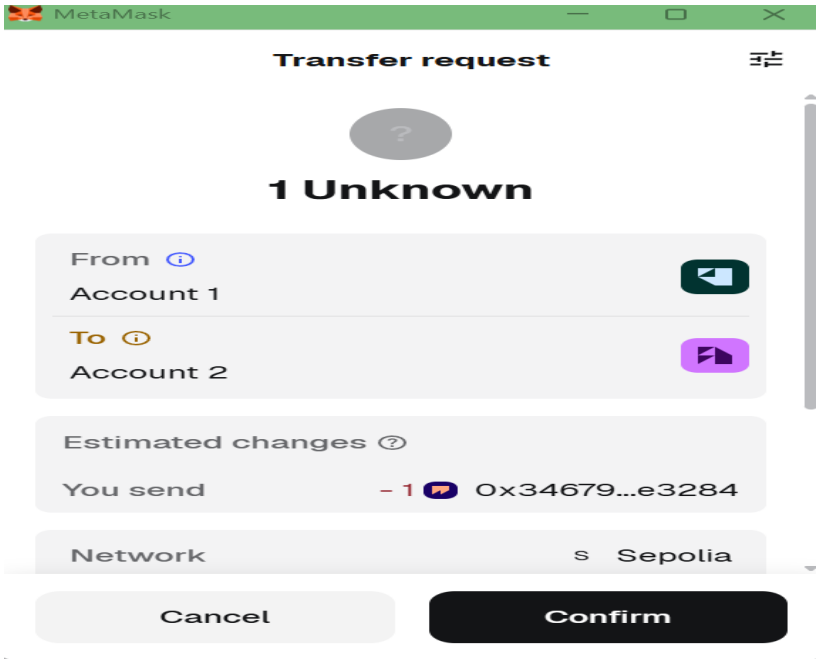


Transaction Request

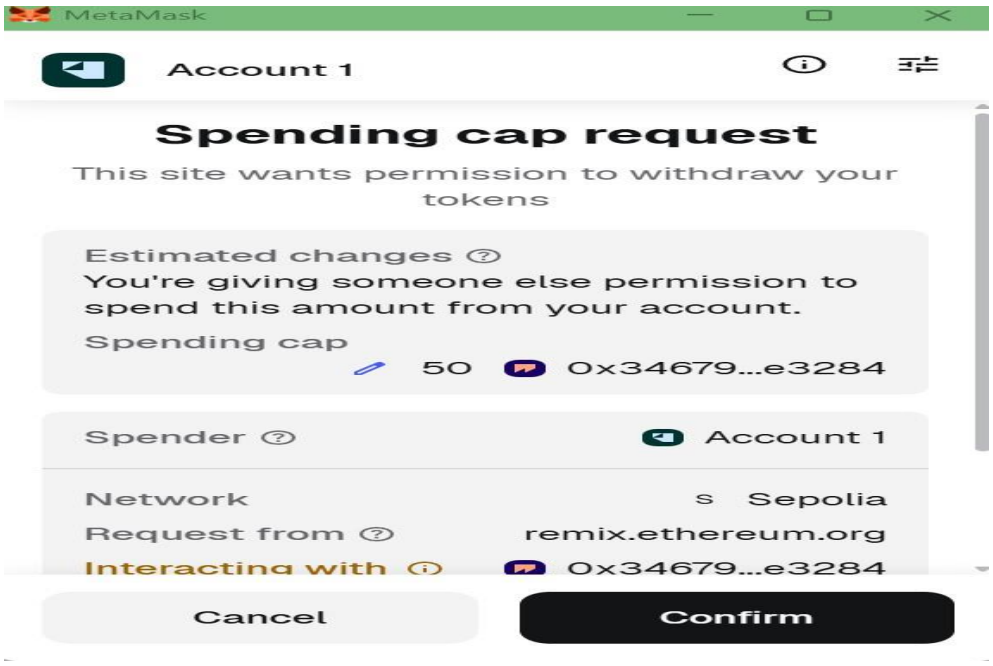


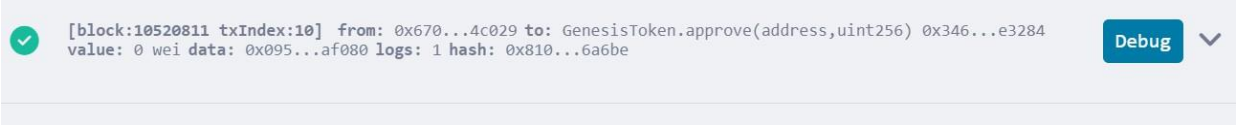
Confirmation on Metamask

Transferred 1 token to another account via metamask

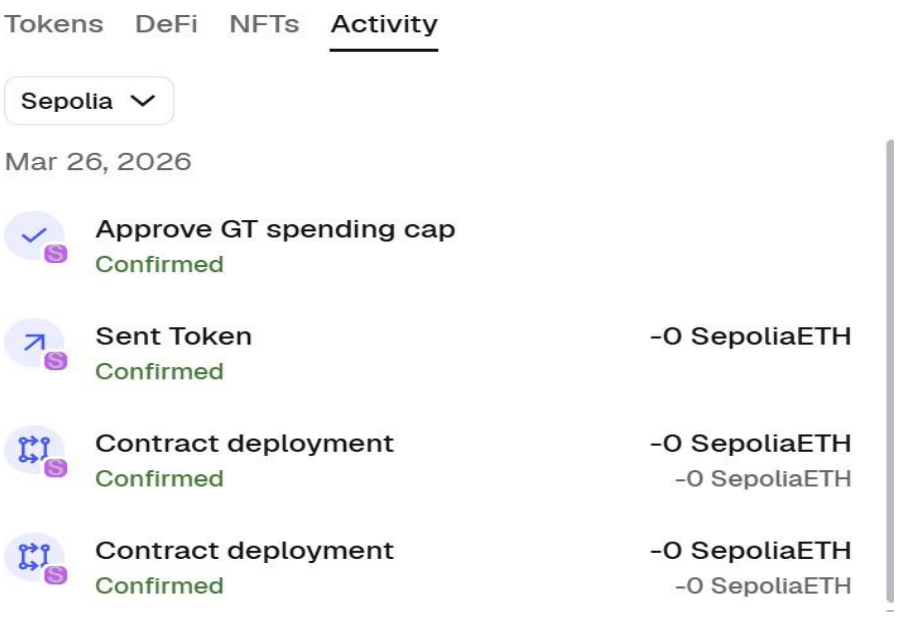


Approved the spending of tokens





Contract details on metamask



Conclusion-

The embedding wallet MetaMask was successfully integrated with Remix IDE to interact with smart contracts written in Solidity. Transactions such as deployment and token transfer were executed on the Sepolia Testnet. This experiment demonstrated secure wallet connectivity, transaction signing, and practical implementation of blockchain-based operations.